# MicroProj AOC — Desempenho de Monociclo/Multiciclo e Pipeline em VHDL

## 1. Enunciado

Cada trio implementará em VHDL três organizações de processador para a mesma ISA, executará nelas o mesmo traço de instruções e medirá quanto o pipeline realmente aceleram a execução. As versões com pipeline não resolvem conflitos: elas os detectam, abortam a execução e informam o tipo, o PC e o ciclo.

### 1.1 Objetivos de aprendizagem

Ao final do MicroProj, você deve ser capaz de:

- **OA1 (Aplicar):** implementar em VHDL um caminho de dados monociclo (ou multiciclo) e um pipeline de 5 estágios.
- **OA2 (Analisar):** identificar e classificar conflitos de dados, estruturais e de controle a partir dos sinais dos registradores entre estágios.
- **OA3 (Aplicar):** medir ciclos, CPI, IPC, tempo de execução, speedup e ocupação a partir da simulação.
- **OA4 (Avaliar):** comparar o desempenho medido com o teórico ideal e explicar as diferenças.
- **OA5 (Criar):** documentar o projeto de forma que outro trio consiga reproduzir seus resultados.

### 1.2 Descrição

Vocês vão construir duas organizações, todas executando a mesma ISA e o mesmo programa:

| Organização | Descrição | Papel na comparação |
|---|---|---|
| O1 — Linha de base | Monociclo **ou** multiciclo, sem pipeline (escolha e justifique) | Referência para o speedup |
| O2 — Pipeline escalar | 5 estágios: IF, ID, EX, MEM, WB; 1 instrução por ciclo | Isola o ganho do pipeline |

O2 e O3 detectam conflitos mas **não os resolvem**: não há forwarding, stall, flush nem predição de desvios. Por isso, o traço principal precisa ser construído pelo trio sem nenhum conflito. Assim, a diferença de desempenho entre O1, O2 e O3 vem só da organização.

### 1.3 Requisitos

**R1 — ISA.** Use MIPS32, como nas aulas e no Patterson & Hennessy. RV32I é aceito se o trio preferir, com as instruções equivalentes da tabela. O subconjunto mínimo obrigatório é:

| Classe | MIPS32 | RV32I equivalente | Formato (MIPS) |
|---|---|---|---|
| Aritmética | add, sub, addi | add, sub, addi | R, R, I |
| Lógica / comparação | and, or, slt | and, or, slt | R |

| | | | |
|---|---|---|---|
| Acesso à memória | lw, sw | lw, sw | I |
| Desvio | beq | beq | I |
| Fim de programa | break | ebreak | — |

Instruções extras (bne, j, andi, ori) são opcionais e devem constar da tabela da ISA do relatório. A instrução de fim (break/ebreak) não conta como instrução útil no cálculo do CPI. Escritas no registrador zero ($zero/x0) não geram dependência.

**R2 — Organizações.** As duas organizações usam memórias de instruções e de dados separadas por padrão. O banco de registradores escreve na primeira metade do ciclo e lê na segunda, como no livro.

**R3 — Interface obrigatória.** As duas organizações expõem a mesma entidade de topo, para que um único testbench sirva a todas e outro trio consiga reproduzir os testes:

| Porta / genérico | Direção | Tipo | Significado |
|---|---|---|---|
| PROG_FILE | genérico | string | Arquivo .hex com o programa (uma palavra de 32 bits por linha) |
| MEM_UNIFICADA | genérico | boolean | true: IF e MEM compartilham uma porta de memória (só para demonstrar conflito estrutural em O2) |
| clk, rst | entrada | std_logic | Relógio e reset síncrono ativo em 1 |
| done | saída | std_logic | 1 quando a instrução de fim conclui; o processador congela |
| hazard_abort | saída | std_logic | 1 quando um conflito é detectado; o processador congela |
| hazard_type | saída | slv(1 downto 0) | 00 nenhum, 01 dados, 10 estrutural, 11 controle |
| hazard_pc | saída | slv(31 downto 0) | PC da instrução envolvida (ver R4) |
| hazard_cycle | saída | unsigned(31 downto 0) | Ciclo da detecção, contado a partir de 1 após o reset |
| retired | saída | unsigned(1 downto 0) | Quantas instruções úteis concluíram neste ciclo (0, 1 ou 2) |
| stage_valid | saída | slv(9 downto 0) | Bit de validade de cada estágio: bits 4..0 = via 0 (WB..IF), bits 9..5 = via 1. O1 usa só o bit 0 |
| dbg_reg_addr | entrada | slv(4 downto 0) | Endereço de registrador para leitura de depuração |
| dbg_reg_data | saída | slv(31 downto 0) | Conteúdo do registrador, leitura combinacional |

**R4 — Unidade de detecção de conflitos (O2).** Ao detectar um conflito, o processador ativa hazard_abort, preenche hazard_type, hazard_pc e hazard_cycle e para de alterar PC, registradores entre estágios, banco de registradores e memória. Os sinais ficam estáveis até o próximo reset. A detecção deve ocorrer **antes** de a instrução prejudicada produzir efeito errado. Os casos a detectar são:

| Tipo | Quando ocorre | hazard_pc aponta para |
|---|---|---|
| Dados | Uma instrução lê um registrador que uma instrução anterior ainda não escreveu no momento da leitura. Em O3, inclui dependência entre as duas instruções do mesmo pacote (RAW) e as duas escrevendo o mesmo | A instrução consumidora |

| | | |
|---|---|---|
| | registrador (WAW) | |
| Estrutural | O2 com MEM_UNIFICADA = true: IF e um lw/sw em MEM no mesmo ciclo. | A instrução mais nova, que não obteve o recurso |
| Controle | Um desvio tomado (ou salto) com instruções já buscadas depois dele. O trio define e justifica o estágio em que o desvio é resolvido | O desvio |

Buscar sequencialmente após um beq não tomado não é conflito: o resultado é correto. Prepare-se para justificar essa escolha na defesa.

**R5 — Traços de execução.** Cada trio entrega:

- **T0 (traço limpo):** um programa com pelo menos 30 instruções úteis, sem conflitos em O1 e O2, com todas as classes da ISA. Ele é o mesmo nas três organizações. Prefira instruções independentes a NOPs para afastar dependências. Se usar NOPs, informe o CPI com e sem contá-los.
- **TD, TE, TC (traços com conflito):** um traço curto para cada tipo (dados, estrutural, controle) que provoca o aborto em O2.
- **S (série de tamanhos):** variações de T0 com 5, 10, 20, 50, 100 e 200 instruções úteis, para o gráfico de speedup por tamanho. Podem ser geradas por script.

Cada traço vem com o fonte em assembly, o .hex gerado (MARS ou RARS: Dump Memory em hexadecimal) e um arquivo esperado.txt com os valores finais dos registradores.

**R6 — Métricas.** Para cada organização e cada traço limpo, calculem: ciclos totais, CPI, IPC, tempo de execução (ciclos × período de clock), speedup de O2 sobre O1 comparado ao speedup ideal, efeito do enchimento e esvaziamento do pipeline em traços curtos e longos, e taxa de ocupação por estágio e por via. Contem os ciclos do primeiro ciclo após o reset até o ciclo em que done vale 1, inclusive.

O período de clock de cada organização pode vir de síntese (Yosys, Quartus ou Vivado) ou de uma tabela de atrasos por componente, como a do livro, somando o overhead dos registradores entre estágios. Em qualquer caso, justifiquem o valor adotado e usem o mesmo critério nas três organizações.

**R7 — Ambiente.** VHDL-2008, simulado em GHDL + GTKWave ou ModelSim/Questa. O repositório deve ter um script (Makefile ou .sh) que compila, roda todos os traços e gera a tabela de métricas com um único comando.

## 1.4 Entregáveis

| Entregável | Conteúdo | Esperado |
|---|---|---|
| Repositório Git | Código VHDL das 3 organizações, testbench, traços, scripts, README com instruções de execução | Fim da semana 4 |
| Relatório | PDF de 4 a 10 páginas no modelo SBC, mais apêndices | Fim da semana 4 |
| Avaliação por pares | Rubrica preenchida, com justificativa por critério, sobre o trio designado | Semana 5 |
| Defesa oral | Slides e demonstração ao vivo | Semanas 5 e 6 |

## 1.5 Composição da nota

A nota do MicroProj soma 100%: Relatório 40% (34% avaliação do professor + 6% avaliação por pares) e Defesa oral 60%. A arguição é individual, então integrantes do mesmo trio podem ter notas finais diferentes.

$$N_{final} = 0{,}40 \times (0{,}85 \times N_{prof} + 0{,}15 \times N_{pares}) + 0{,}60 \times N_{defesa}$$

# 2. Cronograma

O desenvolvimento ocupa 4 semanas extraclasse, com um checkpoint curto ao fim de cada semana; avaliação por pares e defesas vêm nas semanas 5 e 6. Os checkpoints não valem nota, mas o professor registra o andamento e pode usar esse registro na arguição.

| Semana | Atividades do trio | Marco (checkpoint) | Evidência a mostrar |
|---|---|---|---|
| 0 (aula de lançamento) | Formar trios, criar o repositório, ler o enunciado | M0 — trio e repositório registrados | Link do repositório enviado |
| 1 | Tabela da ISA; O1 completa; testbench com contagem de ciclos e verificação automática; primeira versão de T0 | M1 — O1 executa T0 corretamente | Saída do testbench sem erros de assert |
| 2 | O2 com registradores entre estágios e bits de validade; unidade de detecção; traços TD, TE e TC | M2 — O2 executa T0 e aborta em TD, TE e TC | Waveform do aborto com hazard_type correto |
| 3 | script que gera a tabela de métricas | M3 — métricas de O1, O2 geradas | Arquivo metricas.csv completo |
| 4 | Período de clock (síntese ou tabela); gráficos; análise; relatório; README | Entrega final (relatório + repositório congelado) | Tag final no repositório |
| 5 | Avaliação por pares do trio designado (prazo de 5 dias); início das defesas | Entrega da avaliação por pares | Rubrica com justificativas |
| 5–6 | Defesas orais conforme escala do professor | Defesa | Slides + simulação ao vivo |

Depois da entrega final, o repositório fica congelado na tag. A demonstração na defesa deve rodar a partir dessa tag.

# 3. Estrutura sugerida do relatório

O relatório segue o modelo SBC de artigos e tem de 4 a 10 páginas, contando do título às referências; apêndices não entram na contagem. Cada seção abaixo indica o que o leitor espera encontrar e uma extensão de referência.

| Seção | Conteúdo esperado | Extensão aproximada |
|---|---|---|
| Título, autores, resumo | Objetivo, o que foi implementado e o principal resultado numérico (ex.: speedup de O2 sobre O1 em T0) | 1 parágrafo |

| | | |
|---|---|---|
| 1. Introdução | Problema, objetivo do MicroProj, organização do texto | 0,5 página |
| 2. Fundamentação | Só o necessário: pipeline, tipos de conflito, emissão múltipla estática, métricas, com citação do Patterson & Hennessy | 0,5 a 1 página |
| 3. Metodologia | ISA implementada (tabela), escolha de O1 (monociclo ou multiciclo) e por quê, regras usadas para montar T0 sem conflitos, ambiente de simulação, critério do período de clock | 1 a 1,5 página |
| 4. Implementação | Caminho de dados de O1 e O2; registradores entre estágios; diagrama da unidade de detecção e as condições lógicas de cada tipo de conflito; decisões de projeto e limitações | 2 a 3 páginas |
| 5. Resultados | Diagramas instrução × ciclo, tabela comparativa de métricas, gráficos, registro dos conflitos detectados com waveforms anotadas | 1,5 a 2,5 páginas |
| 6. Análise crítica | Medido versus ideal e por quê; efeito do tamanho do traço; custo no período de clock; o que mudaria com forwarding, stall e predição; ameaças à validade (ex.: traço artificial, período estimado) | 1 a 1,5 página |
| 7. Conclusão | Resposta direta à pergunta do projeto e lições aprendidas | 0,5 página |
| Referências | Formato SBC | — |
| Apêndice A | Link do repositório, tag final e comando único de reprodução | — |
| Apêndice B | Divisão de tarefas entre os integrantes | — |
| Apêndice C | Declaração de uso de IA (ver seção 8) | — |

Todo número do relatório deve ser reproduzível pelo script do repositório. Figuras e tabelas são numeradas, têm legenda e são citadas no texto.

## 4. Rubrica da avaliação por pares

Cada trio avalia o relatório e o repositório de outro trio, designado pelo professor, atribuindo a cada critério um nível de 0 a 4 com justificativa escrita. Os pesos somam 100%, e a nota por pares (0 a 10) é a soma de peso × nível ÷ 4 × 10. O professor usa a mesma rubrica na sua avaliação do relatório.

| Critério | Peso | 0 — Ausente | 1 — Insuficiente | 2 — Parcial | 3 — Bom | 4 — Excelente |
|---|---|---|---|---|---|---|
| a) Modelagem da organização | 20% | Sem diagramas ou organizações faltando | Diagramas de só uma organização ou incoerentes com o código | As 2 organizações descritas, mas sem registradores entre estágios ou sem a unidade de detecção | 2 organizações, registradores entre estágios e unidade de detecção descritos, com pequenas lacunas | Tudo descrito, coerente com o código, decisões de projeto justificadas |
| b) Codificação em VHDL | 20% | Código não compila | Compila, mas alguma organização falha em T0 | As 2 executam T0; detecção falha em algum tipo de conflito | As 2 executam T0 e detectam os 3 tipos; código pouco modular ou pouco comentado | Tudo funciona; código modular, legível, semântica de aborto conforme R4 |
| c) Cálculo das | 20% | Métricas | Menos da | Todas | Todas corretas e | Todas corretas, |

| | | | | | | |
|---|---|---|---|---|---|---|
| métricas | | ausentes | metade das métricas de R6, ou com erros de cálculo | presentes, com erros pontuais ou período de clock sem justificativa | justificadas; série S incompleta | período justificado, série S completa, números conferem com o script |
| d) Análise crítica dos resultados | 25% | Sem análise | Apenas repete os números | Compara medido e ideal sem explicar as diferenças | Explica as diferenças (enchimento, período de clock, ocupação) | Além disso, discute limitações do traço, ameaças à validade e o que mudaria com forwarding, stall e predição |
| e) Comunicação e reprodutibilidade | 15% | Fora do modelo SBC e sem instruções de execução | Texto confuso; reprodução exige ajuda dos autores | Texto claro; reprodução exige ajustes manuais | Texto claro; resultados reproduzidos com o README e pequenos ajustes | Texto claro e conciso; resultados reproduzidos com um único comando |

Regras da avaliação por pares:

- O trio avaliador **deve executar** o repositório avaliado e informar se conseguiu reproduzir a tabela de métricas.
- Cada critério exige uma justificativa de pelo menos 3 linhas, citando seção, figura ou arquivo específico. Critério sem justificativa não é considerado.
- O professor pode desconsiderar uma avaliação incoerente com as evidências. Nesse caso, os 15% da Parte I passam a vir da avaliação do professor.
- A qualidade da avaliação feita pelo trio não altera a nota do trio avaliado, mas é considerada na arguição (ex.: “qual foi o principal problema que vocês encontraram no projeto do trio X?”).

## 5. Defesa oral – Roteiro Esperado

1. Problema e objetivo.
2. ISA implementada e escolha da linha de base.
3. Caminhos de dados de O1 e O2, com destaque para os registradores entre estágios.
4. Unidade de detecção: condições de cada tipo de conflito e semântica de aborto.
5. Resultados: tabela comparativa, gráficos de CPI, speedup e speedup por tamanho.
6. Análise crítica e conclusão.
7. Demonstração ao vivo a partir da tag final:
    1. Rodar o script único e mostrar a tabela de métricas gerada.
    2. Abrir a waveform de T0 em O2 e mostrar o enchimento e o esvaziamento do pipeline.
    3. Rodar o traço de conflito **sorteado pelo professor** na hora (TD, TE ou TC) e mostrar hazard_abort, hazard_type, hazard_pc e hazard_cycle.
    4. Opcional, a pedido do professor: alterar uma instrução de T0 para criar um conflito e mostrar que ele é detectado.

# 6. Artefatos exigidos

O relatório e a defesa devem conter os artefatos abaixo; a coluna “Onde” indica se vão no relatório (R), na defesa (D) ou no repositório (G).

| Tipo | Artefato | Conteúdo mínimo | Onde |
|---|---|---|---|
| Tabela | ISA implementada | Mnemônico, formato, opcode/funct, operação em RTL | R |
| Tabela | Traço T0 | Endereço, instrução, registradores lidos e escritos, distância até o produtor mais próximo | R, G |
| Tabela | Diagrama instrução × ciclo | Uma por organização, para as 10 primeiras e as 3 últimas instruções de T0 (formato abaixo) | R, D |
| Tabela | Comparativo de métricas | Por organização: ciclos, CPI, IPC, período (ps), tempo (ns), speedup medido, speedup ideal, ocupação | R, D |
| Tabela | Registro de conflitos | Por traço de conflito: tipo esperado, tipo detectado, hazard_pc, hazard_cycle, organização | R, D |
| Diagrama | Caminho de dados | Um por organização; em O2, com registradores entre estágios nomeados (IF/ID, ID/EX, EX/MEM, MEM/WB) | R, D |
| Diagrama | Unidade de detecção | Blocos, sinais de entrada vindos de cada registrador entre estágios e saídas hazard_* | R, D |
| Diagrama | Registradores entre estágios | Lista de campos de cada registrador, com largura em bits | R |
| Waveform | Execução de T0 | clk, PC, stage_valid, retired, done; em O2, o enchimento e o esvaziamento anotados | R, D |
| Waveform | Detecção e aborto | Um por tipo de conflito: os campos que disparam a detecção, hazard_abort e o congelamento do PC, anotados no ciclo exato | R, D |
| Testbench | tb_cpu.vhd | Estrutura da seção 7, reutilizado nas 2 organizações | G |
| Script | run_all | Compila, simula todos os traços e gera metricas.csv | G |
| Gráfico | CPI por organização | Barras, uma por organização, para T0 | R, D |
| Gráfico | Speedup por organização | Barras de speedup medido ao lado do ideal | R, D |
| Gráfico | Speedup × tamanho do traço | Uma linha por organização (O2) sobre a série S; eixo x em escala logarítmica | R, D |

## 6.1 Formato do diagrama instrução × ciclo

Cada linha é uma instrução e cada coluna um ciclo; a célula mostra o estágio. O exemplo abaixo mostra só o formato, com instruções independentes genéricas:

| Instrução | C1 | C2 | C3 | C4 | C5 | C6 | C7 |
|---|---|---|---|---|---|---|---|
| I1 | IF | ID | EX | MEM | WB | | |
| I2 | | IF | ID | EX | MEM | WB | |
| I3 | | | IF | ID | EX | MEM | WB |

## 6.2 Anotação das waveforms

Toda waveform no relatório deve mostrar o tempo ou o número do ciclo no eixo, ter os sinais agrupados por estágio e trazer marcadores com texto no ciclo relevante (ex.: “C12: lw $t1 em EX e add que lê $t1 em ID → conflito de dados”). Use valores em hexadecimal para PC e instruções.

# 7. Esqueleto de testbench em VHDL

Um único testbench serve às duas organizações, porque todas expõem a interface de R3. Ele gera clock e reset, carrega o programa pelo genérico PROG_FILE, conta ciclos, instruções e ocupação, verifica o desfecho com assert/report e grava uma linha em metricas.csv. Os trechos marcados com TODO são do trio.

```
-- tb_cpu.vhd : testbench único para O1 e O2 (VHDL-2008)
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use std.textio.all;

entity tb_cpu is
  generic (
    PROG_FILE     : string   := "tracos/t0.hex";          -- programa (.hex)
    ESPERADO_FILE : string   := "tracos/t0_esperado.txt"; -- linhas "reg
valor_hex"
    ORG_NOME      : string   := "O2";      -- rótulo usado na saída
    NUM_SLOTS     : positive := 5;         -- O1 = 1, O2 = 5
    MEM_UNIFICADA : boolean  := false;     -- true só no traço TE de O2
    ESPERA_ABORTO : boolean  := false;     -- true nos traços TD, TE e TC
    TIPO_ESPERADO : natural  := 0;         -- 1 dados, 2 estrutural, 3
controle
    T_CLK         : time     := 10 ns;     -- período de SIMULAÇÃO, não o
real
    MAX_CICLOS    : natural  := 10_000     -- proteção contra laço infinito
  );
end entity;

architecture sim of tb_cpu is
  signal clk          : std_logic := '0';
  signal rst          : std_logic := '1';
  signal fim_sim      : boolean   := false;
  signal done         : std_logic;
  signal hazard_abort : std_logic;
  signal hazard_type  : std_logic_vector(1 downto 0);
  signal hazard_pc    : std_logic_vector(31 downto 0);
  signal hazard_cycle : unsigned(31 downto 0);
  signal retired      : unsigned(1 downto 0);
  signal stage_valid  : std_logic_vector(9 downto 0);
  signal dbg_reg_addr : std_logic_vector(4 downto 0) := (others => '0');
  signal dbg_reg_data : std_logic_vector(31 downto 0);
begin
```

```
  -- 1. Geração de clock: para quando fim_sim = true
  clk <= not clk after T_CLK / 2 when not fim_sim else '0';

  -- 2. Processador sob teste. Cada organização fica em sua pasta, com a
  --    entidade cpu_top, e é compilada em uma biblioteca de trabalho
separada.
  dut : entity work.cpu_top
    generic map (PROG_FILE => PROG_FILE, MEM_UNIFICADA => MEM_UNIFICADA)
    port map (
      clk => clk, rst => rst, done => done,
      hazard_abort => hazard_abort, hazard_type => hazard_type,
      hazard_pc => hazard_pc, hazard_cycle => hazard_cycle,
      retired => retired, stage_valid => stage_valid,
      dbg_reg_addr => dbg_reg_addr, dbg_reg_data => dbg_reg_data
    );

  -- 3. Processo principal
  principal : process
    file     f_esp    : text;
    file     f_csv    : text;
    variable l        : line;
    variable st       : file_open_status;
    variable ciclos   : natural := 0;
    variable instrs   : natural := 0;
    variable ocupados : natural := 0;
    variable erros    : natural := 0;
    variable reg_num  : integer;
    variable valor    : std_logic_vector(31 downto 0);
    variable cpi, ipc, ocup : real := 0.0;
  begin
    -- 3.1 Reset síncrono durante 2 ciclos
    rst <= '1';
    wait until rising_edge(clk);
    wait until rising_edge(clk);
    rst <= '0';

    -- 3.2 Execução: amostra na borda de DESCIDA, no meio do ciclo,
    --     quando as saídas do processador já estão estáveis
    loop
      wait until falling_edge(clk);
      ciclos := ciclos + 1;
      instrs := instrs + to_integer(retired);
      for i in 0 to NUM_SLOTS - 1 loop
        if stage_valid(i) = '1' then
          ocupados := ocupados + 1;
        end if;
      end loop;
      exit when done = '1' or hazard_abort = '1' or ciclos = MAX_CICLOS;
    end loop;

    -- 3.3 Desfecho: aborto, fim normal ou tempo esgotado
    if hazard_abort = '1' then
      report ORG_NOME & ": ABORTO tipo=" & to_string(hazard_type) &
```

```
              " pc=0x" & to_hstring(hazard_pc) &
              " ciclo=" & integer'image(to_integer(hazard_cycle));
      assert ESPERA_ABORTO
        report "Aborto inesperado: o traço deveria ser livre de conflitos"
        severity error;
      assert to_integer(unsigned(hazard_type)) = TIPO_ESPERADO
        report "Tipo de conflito diferente do esperado" severity error;
      assert to_integer(hazard_cycle) = ciclos
        report "hazard_cycle difere da contagem do testbench" severity
warning;
      -- TODO: conferir também hazard_pc com o PC esperado do traço

    elsif done = '1' then
      assert not ESPERA_ABORTO
        report "Conflito esperado não foi detectado" severity error;

      -- 3.4 Verificação automática: compara registradores com o esperado
      file_open(st, f_esp, ESPERADO_FILE, read_mode);
      assert st = open_ok
        report "Não foi possível abrir " & ESPERADO_FILE severity failure;
      while not endfile(f_esp) loop
        readline(f_esp, l);
        next when l'length = 0;      -- ignora linhas vazias
        next when l(1) = '#';        -- ignora comentários
        read(l, reg_num);            -- ex.: "9 0000002A"
        hread(l, valor);
        dbg_reg_addr <= std_logic_vector(to_unsigned(reg_num, 5));
        wait for 1 ns;               -- leitura combinacional estabiliza
        if dbg_reg_data /= valor then
          erros := erros + 1;
          report "Reg " & integer'image(reg_num) & ": esperado 0x" &
                 to_hstring(valor) & ", obtido 0x" & to_hstring(dbg_reg_data)
            severity error;
        end if;
      end loop;
      file_close(f_esp);
      -- TODO (opcional): verificar também posições da memória de dados

    else
      report ORG_NOME & ": tempo esgotado em " & integer'image(MAX_CICLOS) &
             " ciclos" severity failure;
    end if;

    -- 3.5 Métricas: uma linha por execução em metricas.csv
    --     org;programa;ciclos;instrucoes;CPI;IPC;ocupacao;erros
    if instrs > 0 then
      cpi := real(ciclos) / real(instrs);
      ipc := real(instrs) / real(ciclos);
    end if;
    ocup := real(ocupados) / real(ciclos * NUM_SLOTS);
    file_open(st, f_csv, "metricas.csv", append_mode);
    write(l, ORG_NOME & ";" & PROG_FILE & ";" & integer'image(ciclos) & ";" &
             integer'image(instrs) & ";" & real'image(cpi) & ";" &
```

```
            real'image(ipc) & ";" & real'image(ocup) & ";" &
            integer'image(erros));
    writeline(f_csv, l);
    file_close(f_csv);

    -- 3.6 Encerramento
    report ORG_NOME & ": fim da simulação, " & integer'image(ciclos) &
           " ciclos, " & integer'image(erros) & " erro(s)";
    fim_sim <= true;
    std.env.stop;
    wait;
  end process;

end architecture;
```

Exemplo de execução no GHDL, compilando cada organização em uma pasta de trabalho própria:

```
ghdl -a --std=08 --workdir=build/o2 o2/*.vhd tb/tb_cpu.vhd
ghdl -e --std=08 --workdir=build/o2 tb_cpu
ghdl -r --std=08 --workdir=build/o2 tb_cpu \
     -gPROG_FILE=tracos/td.hex -gESPERA_ABORTO=true -gTIPO_ESPERADO=1 \
     --wave=build/o2_td.ghw
```

Observações para os trios:

- O contador interno que gera hazard_cycle deve seguir a mesma convenção do testbench: o ciclo 1 é o primeiro após a liberação do reset. O assert de aviso em 3.3 ajuda a encontrar diferenças.
- T_CLK só serve para a simulação. O tempo de execução do relatório usa o período justificado em R6.
- O formato do esperado.txt é número do registrador em decimal e valor em hexadecimal, separados por espaço.

# 8. Uso de IA e integridade acadêmica

Ferramentas de IA generativa são permitidas como apoio ao estudo e à depuração, mas o projeto do processador e da unidade de detecção deve ser do trio, e todo uso precisa ser declarado. A arguição individual existe justamente para verificar que cada integrante domina o que foi entregue.

| Uso | Situação | Exemplos |
|---|---|---|
| Permitido | Livre, com declaração | Explicar conceitos (pipeline, conflitos, emissão múltipla); interpretar mensagens de erro do GHDL; revisar a redação do relatório; gerar scripts auxiliares (Makefile, geração da série S, gráficos) |
| Permitido com cautela | Declarar e revisar linha a linha | Sugestões pontuais de sintaxe VHDL (ex.: leitura de arquivo .hex); revisão de um trecho escrito pelo trio |
| Não permitido | Pode zerar o critério afetado | Gerar o caminho de dados, os registradores entre estágios ou a unidade de detecção; gerar análise de resultados que o trio não produziu; inventar números, waveforms ou referências |

## 8.1 Declaração de uso de IA (Apêndice C)

O apêndice informa, para cada uso: a ferramenta e a versão, a finalidade, os prompts principais (ou um resumo fiel), o trecho do projeto afetado e como o trio verificou o resultado. Um projeto sem nenhum uso de IA declara isso explicitamente.

## 8.2 Regras de integridade

- O histórico de commits deve mostrar a evolução do projeto pelos três integrantes. Um repositório com o código inteiro em poucos commits finais será investigado na arguição.
- Trios podem discutir ideias entre si, mas não compartilhar código, traços ou textos. Na avaliação por pares, é proibido copiar trechos do projeto avaliado.
- Toda fonte externa (livro, tutorial, repositório) usada como base deve ser citada no relatório e no código.
- Se um integrante não conseguir explicar um trecho que declarou ter feito, a nota individual de arguição reflete isso, independentemente da nota do trio.
- Casos de plágio ou uso não declarado seguem as normas da UFRR para infrações acadêmicas.